

E.E.S.T. N.º 1 — “Eduardo Ader”
Vicente López
Laboratorio de Programación — 5º Año 3º Grupos A-B
Miércoles y jueves de 15:15 a 17:15 Hs

ACTIVIDAD 4: MANIPULACIÓN DE CADENAS Y PUNTEROS EN C/C++

Control de acceso por edad mediante punteros y desreferenciación en memoria RAM

Estudiante: TU NOMBRE Y APELLIDO — Legajo: _____

Curso: 5º 3º A-B | **Profesor:** Mansilla Muñoz York Elías | **Fecha de entrega:** 17 de Septiembre de 2026

Repositorio GitHub: https://github.com/TU_USUARIO/CADENASPUNTEROS (reemplazar por URL real)

Entorno: Windows 10/11 + Visual Studio Code + TDM-GCC g++ 10.3.0

[Informe Técnico — Normas APA 7ª edición](#)

1º Cuatrimestre — 2026

Docente: @docentedeclasesdeapoyo

Tabla de Contenidos

1. Introducción y Objetivos
2. Marco Teórico — Glosario Técnico
3. Desarrollo — Análisis de main.cpp y memoria RAM
4. Cuestionario Técnico de Control (3 preguntas)
5. Evidencias — Figuras APA v7
6. Conclusiones y Referencias

1. Introducción y Objetivos

Para comprender la programación de bajo nivel es necesario analizar qué ocurre en la memoria RAM, organizada como casilleros numerados con direcciones físicas únicas en formato hexadecimal (0x...). El objetivo de la actividad es desarrollar un sistema básico de control de acceso por edad en C++ manipulando los datos directamente en la RAM mediante el uso de punteros (`int* p = &edad;`) y desreferenciación (`*p`).

El programa solicita la edad al usuario, la almacena desreferenciando el puntero (`cin >> *p`) y decide el acceso con `if (*p >= 18)`, mostrando tanto el valor como la dirección física en RAM (`cout << p`).

2. Marco Teórico — Glosario Técnico

Memoria RAM: Espacio físico de almacenamiento temporal donde residen las variables durante la ejecución.

Dirección Hexadecimal: Representación en base 16 (prefijo 0x) que identifica un casillero exacto de la RAM.

Puntero (Pointer): Variable que guarda la dirección de memoria de otra variable.

Operador de Dirección (&): Devuelve la dirección física en RAM donde está guardada una variable. Ej: `&edad;`

Desreferenciar (*): Operación para leer o modificar el valor contenido en la dirección apuntada. Ej: `*p = 20`.

Cadena estilo C: Arreglo de char terminado en carácter nulo `\0`. Se recorre con puntero `const char *strTexto`. Funciones asociadas: `atoi()` y `strcmp()`.

3. Desarrollo — Análisis de main.cpp

Estructura del proyecto (exigida por la consigna):

```
CADENASPUNTEROS/  
■■■■ .gitignore  
■■■■ LICENSE  
■■■■ README.md  
■■■■ Docs/InformeProyectoCADENASPUNTEROS.pdf  
■■■■ src/main.cpp  
■■■■ Capturas/  
■■■■ ejecucion_punteros.png  
■■■■ esquema_ram.png
```

Fragmento clave de `src/main.cpp:14-15`:

```
int edad = 0;  
int* p = &edad; // p guarda la dirección física de edad
```

La variable **edad** ocupa un casillero en RAM (ej: 0x61fe1c). El puntero **p** ocupa otro casillero distinto (ej: 0x61fe18) cuyo contenido es 0x61fe1c. La desreferenciación ***p** es un acceso indirecto: 've a la dirección que guarda p y operá sobre ese casillero'. Por eso `cin >> *p` escribe sobre **edad** y `cout << *p` lee su valor, mientras que `cout << p` muestra la dirección.

Compilación y ejecución verificada:

```
g++ src/main.cpp -o src/cadenaspunteros.exe
.\src\cadenaspunteros.exe
// Ingresar 20 → [ACCESO APROBADO] 0x...
// Ingresar 15 → [ACCESO RECHAZADO] 0x...
```

4. Cuestionario Técnico de Control

Pregunta 1: Diferencia entre &y * — ¿Qué imprime cout << p; vs cout << *p;?

Respuesta: &edad (o el valor de p) es la **dirección**: un hexadecimal tipo 0x61fe1c (en 64 bits ej: 0x2f2d5ff844 visto en la ejecución de prueba). *p es el **contenido** de esa dirección: el entero almacenado (ej: 20). Por eso cout << p imprime la dirección física en RAM (útil para debug de punteros) y cout << *p imprime el valor lógico de la variable edad. En memoria: p == &edad es verdad, y *p == edad es verdad. Mostrar solo p sin desreferenciar no muestra la edad.

Pregunta 2: Error Crítico al Guardar Entrada — ¿Por qué es obligatorio cin >> *p;?

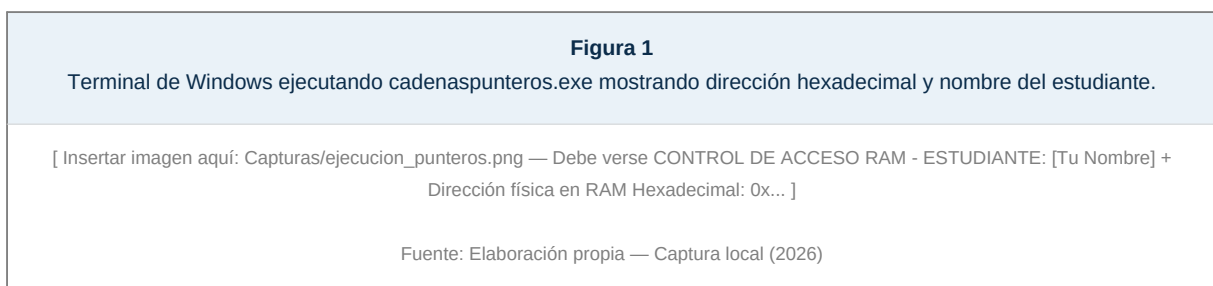
Respuesta: p es de tipo int* (dirección). *p es de tipo int (valor). cin >> p intentaría extraer un entero hacia una variable que almacena direcciones, lo cual es un error de tipos: no existe sobrecarga de operador >> para int* en ese contexto y, si compilara, sobrescribiría la dirección apuntada, rompiendo el vínculo con edad y causando comportamiento indefinido (puntero colgante / segmentation fault). En cambio cin >> *p desreferencia: el operador * resuelve la dirección y devuelve una referencia int& al casillero original de edad, por lo que la lectura modifica directamente esa casilla. Es equivalente a cin >> edad pero demostrando acceso indirecto por puntero.

Pregunta 3: Formato Hexadecimal — ¿Por qué el SO usa hexadecimal para direcciones?

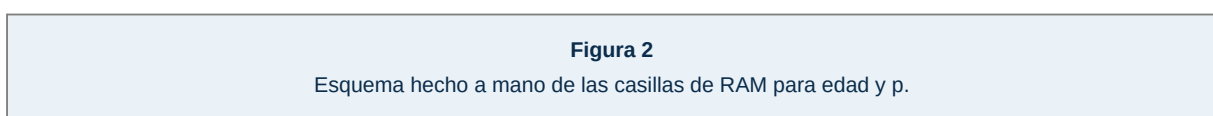
Respuesta: Tres razones técnicas: **1) Compacidad:** 1 dígito hex = 4 bits. Una dirección de 32 bits son 8 dígitos hex (0x61FE1C) vs 10 dígitos decimales; en 64 bits la ventaja es mayor. **2) Alineación binaria directa:** al ser base 2, cada dígito hex mapea exactamente a un nibble, lo que permite leer máscaras, offsets y alineación de bytes sin conversión. Ej: 0xFF = 11111111 = 255. **3) Legibilidad de bajo nivel:** el hardware direcciona por bytes/páginas potencias de 2 (0x1000 = 4096). En hex los límites de página y los desbordamientos son visibles (0x0F + 1 = 0x10) mientras que en decimal quedan ocultos. Por eso depuradores, dumps de RAM y el operador << para punteros usan 0x.

5. Evidencias — Figuras APA v7

Para cumplir la entrega, reemplazar los recuadros siguientes por las capturas reales en formato PNG.



Nota. Captura tomada en PowerShell con g++ src/main.cpp -o src/cadenaspunteros.exe y ejecución con entradas 20 y 15 para mostrar ambos casos (APROBADO/RECHAZADO).



[Insertar imagen aquí: Capturas/esquema_ram.png — Dibujar dos casillas: edad (contiene valor ej: 20, dirección 0x61FE1C) y p (contiene 0x61FE1C, dirección propia 0x61FE18), flecha APUNTA A y flecha DESREFERENCIA (*) de vuelta. Foto legible en .png]

Fuente: Elaboración propia — Captura local (2026)

Nota. Fotografía del cuaderno/carpetas con diagrama rotulado. Referenciar como Figura 2 en el texto.

6. Conclusiones y Referencias

El uso de punteros permite control explícito sobre la RAM: & obtiene la dirección, * accede al valor. Comprender esta dualidad evita errores críticos como escribir sobre la dirección en lugar del valor. La notación hexadecimal es el estándar operativo por su correspondencia directa con el binario subyacente. El programa cumple el objetivo de control de acceso operando de forma segura sobre memoria.

Referencias (APA v7):

cplusplus.com. (s. f.). Pointers. *cplusplus.com Reference*. <https://cplusplus.com/doc/tutorial/pointers/>

Microsoft Learn. (2024). Operadores de dirección y desreferenciación en C++. *Microsoft C++ Documentation*. <https://learn.microsoft.com/cpp/cpp/addresses-and-indirections>

GeeksforGeeks. (s. f.). Pointers in C++. <https://www.geeksforgeeks.org/pointers-in-c-and-c-set-1-introduction-arithmetic-and-array/>

Anexo: Código fuente completo disponible en src/main.cpp y repositorio GitHub declarado en carátula. Compilación verificada con g++ 10.3.0 (TDM64).